

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 5 SINGLE LINKED LIST



Disusun Oleh :

DIKA GIOWANDA

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, MEI 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan rahmat, hidayah, dan karunia-Nya sehingga laporan praktikum Struktur Data tentang Single Linked List ini dapat diselesaikan dengan baik dan tepat waktu. Sholawat serta salam senantiasa tercurah kepada Nabi Muhammad SAW, keluarga, sahabat, dan seluruh pengikutnya hingga akhir zaman.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban, refleksi, serta dokumentasi atas kegiatan praktikum yang telah dilaksanakan, dengan fokus pada penerapan konsep struktur data menggunakan *Single Linked List*. *Single Linked List* merupakan salah satu struktur data linear yang banyak digunakan karena kemudahannya dalam mengelola, menyimpan, dan memanipulasi data secara dinamis dengan menggunakan node yang saling terhubung melalui *pointer*. Dalam praktikum ini, penulis mempelajari operasi-operasi dasar pada *Single Linked List* seperti pembuatan node, penambahan elemen di awal (*insert head*), penambahan elemen di akhir (*insert tail*), penambahan elemen di posisi tertentu, penghapusan elemen, pencarian data, serta traversing untuk menampilkan seluruh isi list.

Penyusunan laporan ini tidak mungkin terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih yang tulus kepada dosen pembimbing yang telah memberikan arahan, bimbingan, dan masukan yang berharga selama proses praktikum, serta kepada rekan-rekan mahasiswa yang turut berkontribusi dalam diskusi dan kolaborasi.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca, khususnya dalam memahami dasar-dasar dan aplikasi struktur data *Single Linked List*.

Akhir kata, penulis berharap semoga ilmu yang diperoleh melalui praktikum ini dapat bermanfaat dan diterapkan dengan baik.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB 1 PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Tujuan Praktikum.....	1
1.3 Persyaratan Praktikum.....	1
1.4 Tempat dan Waktu Praktikum.....	2
1.5 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN PRAKTIKUM.....	3
2.1 Pengertian Pemrograman Single Linked List.....	3
2.3 Langkah-Langkah Pengerjaan Praktikum Pekan 5.....	4
BAB 3 PENUTUP.....	19
3.1 Kesimpulan.....	19
3.2 Saran.....	19
DAFTAR PUSTAKA.....	20
LAPORAN TUGAS.....	21

BAB 1

PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan konsep pemrograman Java. Kegiatan ini tidak hanya menekankan pada penguasaan teori, tetapi juga pada latihan penyusunan kode program, pengujian, hingga analisis hasil eksekusi. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual dengan kemampuan teknis pemrograman.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar pemrograman Struktur Data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Java.
3. Memahami tipe data dasar hingga kompleks, termasuk konsep penyimpanan data.
4. Mampu menerapkan struktur data linear (*Linked List, Stack, Queue*) dan dalam bahasa pemrograman
5. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
6. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
7. Mengetahui dan mengaplikasikan *ADT, ArrayList, Stack, Queue, Single Linked List, Double Linked List, Sorting 1 (Insertion, selection, bubble), Sorting 2 (shell, quick, merge), dan Searching (BFS dan DFS)*.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang *Java Development Kit (JDK)* dan *Integrated Development Environment (IDE)* atau *Eclips* yang direkomendasikan.
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

1.5 Manfaat Praktikum

Manfaat praktikum Struktur Data meliputi kemampuan memahami cara menyimpan data secara terstruktur, rapi, dan efisien, sehingga mengurangi penggunaan memori dan mempercepat akses data. Meningkatkan efisiensi penyimpanan dan manipulasi data, mengoptimalkan kecepatan algoritma, serta mempermudah penyelesaian masalah pemrograman yang kompleks. Selain itu, praktikum ini juga membantu membangun fondasi untuk mengembangkan aplikasi lintas platform yang lebih kompleks di berbagai bidang teknologi.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian Pemrograman Single Linked List

Single Linked List adalah struktur data linear dinamis yang terdiri dari serangkaian node, di mana setiap node menyimpan data dan satu pointer yang menunjuk ke node berikutnya secara satu arah. Node terakhir menunjuk ke *NULL*, menandakan akhir list. Ini digunakan untuk manajemen memori yang efisien karena ukuran tidak statis.

Contoh Penggunaan *Single Linked List* di Praktikum Struktur Data:

- a. Implementasi Antrian (Queue): Mengelola antrian sistem, seperti antrian printer atau customer service.
- b. Manajemen Playlist Musik: Menambahkan atau menghapus lagu secara efisien (lagu berikutnya).
- c. Struktur Data Stack: Implementasi tumpukan (*Last In First Out (LIFO)*).
- d. Manajemen Memori: Alokasi memori yang dinamis untuk data yang sering bertambah/berkurang.
- e. Sistem Navigasi: Skenario yang hanya membutuhkan penelusuran ke depan.

Karakteristik Utama:

1. Dinamis: Ukuran dapat bertambah atau berkurang saat program berjalan.
2. Satu Arah: Navigasi hanya bisa maju (*forward*).
3. Pointer: Menggunakan pointer untuk menghubungkan data yang lokasinya tidak bersebelahan di memori.
4. Operasi: Penambahan/penghapusan data di awal lebih efisien dibanding *array*, tetapi pencarian (akses) lambat ($O(n)$).

Dua komponen utama:

- Data (nilai yang disimpan) dan *Pointer* next (alamat node berikutnya).

- Node terakhir menunjuk ke *NULL* (penanda akhir). [1, 2, 3, 4, 5].

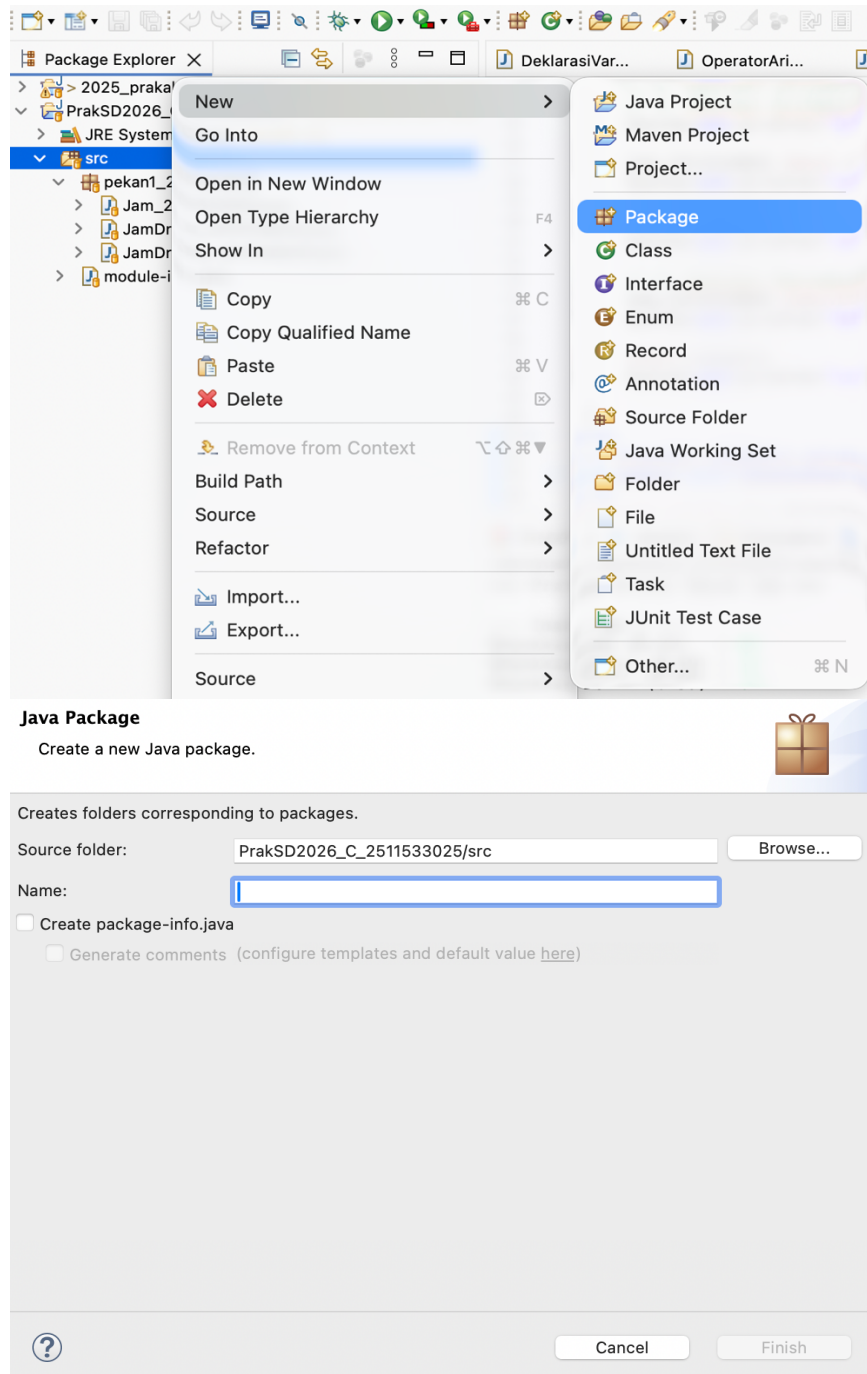
Berikut adalah detail komponennya:

- Node (Simpul): Unit dasar yang menyimpan elemen data dan pointer.
- Data (Info): Bagian dari node yang menyimpan nilai atau informasi yang diinginkan.
- Pointer next (Link/Address): Bagian dari node yang berisi alamat memori node berikutnya, hanya satu arah.
- Head: Pointer yang menunjuk ke node pertama dalam linked list.
- Tail/Null: Penanda node terakhir yang menunjuk ke *NULL*, menandakan akhir dari list.

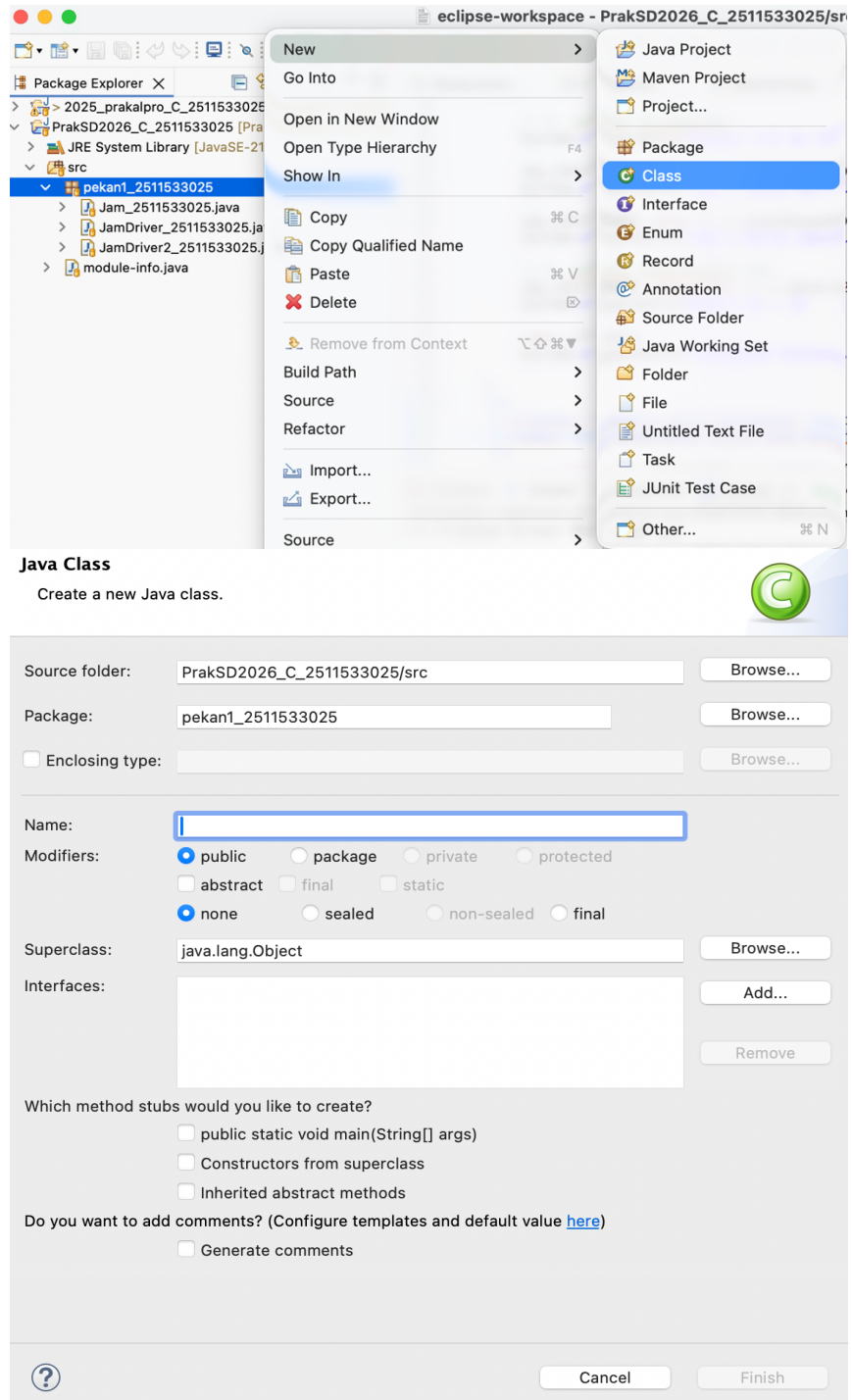
2.2 Langkah-Langkah Pengerjaan Praktikum Pekan 5

Sebelum ke langkah-langkah pengerjaan contoh proyek, saya akan menjelaskan terlebih dahulu gimana caranya menambahkan proyek nya terlebih dahulu :

1. Klik kanan pada folder src, pilih new, pilih package. Nanti akan muncul halaman Java Package, cukup masukan nama, nama wajib no spasi. Klik finish.



2. Klik kanan pada folder package, pilih new, pilih class. Nanti akan muncul halaman Java Class, cukup masukan nama class sesuai proyek yang akan dibuat, nama wajib no spasi. Klik finish.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru.

- A. Baiklah karena proses menambahkan proyek sudah dijelaskan, selanjutnya langkah-langkah pengerjaan proyek pekan 5:

a. Class NodeSLL_2511533025

1. Kode ini mendefinisikan sebuah kelas Java bernama NodeSLL_2511533025 yang berfungsi sebagai struktur dasar atau unit terkecil (node) untuk membangun Single Linked List. Di dalam kelas ini terdapat dua variabel utama, yaitu data_3025 bertipe integer untuk menyimpan nilai data dan next_3025 yang merupakan sebuah referensi atau pointer ke objek node berikutnya dalam daftar. Selain itu, terdapat sebuah konstruktor yang bertugas menginisialisasi setiap node baru dengan mengisi data_3025 sesuai nilai yang dimasukkan dan mengatur next_3025 ke nilai null, menandakan bahwa node tersebut awalnya belum terhubung dengan node manapun.

```
1 package Pekan5_2511533025;
2
3 public class NodeSLL_2511533025 {
4     // node bagian data
5     int data_3025;
6     // pointer ke node berikutnya
7     NodeSLL_2511533025 next_3025;
8     // Konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511533025(int data)
10    {
11        this.data_3025 = data;
12        this.next_3025 = null;
13    }
14
15 }
```

b. Class TambahSLL_2511533025

Class ini merupakan sambungan dari class Jam agar kode dapat berjalan yaitu mengeluarkan output.

1. Kode ini mendefinisikan sebuah metode statis bernama insertAtFront yang berfungsi untuk menyisipkan sebuah simpul (node) baru ke posisi paling depan dalam struktur data Single Linked List. Pertama-tama, metode ini membuat objek baru dari kelas NodeSLL_2511533025 dengan menyimpan nilai data yang diterima melalui parameter value_3025. Selanjutnya, alamat dari simpul baru tersebut dihubungkan ke simpul yang saat ini menjadi kepala list (head_3025) melalui atribut next_3025, sehingga simpul baru tersebut kini berada di depan simpul lama. Terakhir, metode mengembalikan simpul baru tersebut sebagai kepala (head) list yang baru.

```
1 package Pekan5_2511533025;
2 public class TambahSLL_2511533025 {
3     public static NodeSLL_2511533025 insertAtFront(NodeSLL_2511533025 head_3025, int value_3025) {
4         NodeSLL_2511533025 new_node = new NodeSLL_2511533025(value_3025);
5         new_node.next_3025 = head_3025;
6         return new_node;
7     }
8 }
```

2. Kode tersebut untuk menambahkan elemen baru ke akhir sebuah Single Linked List (SLL). Fungsi `insertAtEnd` ini menerima dua parameter, yaitu `head` (titik awal list) dan `value` (nilai yang ingin dimasukkan), kemudian langkah pertamanya adalah membuat objek node baru (`newNode`) yang menampung nilai tersebut. Selanjutnya, terdapat pengecekan kondisi pada baris ke-13: jika list masih kosong (`head == null`), maka node baru tersebut secara otomatis akan menjadi elemen pertama sekaligus terakhir.

```
8 // fungsi menambahkan node di akhir SLL
9 public static NodeSLL_2511533025 insertAtEnd(NodeSLL_2511533025 head_3025, int value_3025) {
10 // buat sebuah node dengan sebuah nilai
11 NodeSLL_2511533025 newNode = new NodeSLL_2511533025(value_3025);
12 // jika list kosong maka node jadi head
13 if (head_3025 == null) {
14 return newNode;
15 }
```

3. Kode ini untuk menambahkan sebuah simpul baru (`newNode`) ke bagian akhir dari sebuah Single Linked List. Proses dimulai dengan menginisialisasi variabel sementara `last` dari posisi `head`, kemudian melakukan penelusuran (traversal) menggunakan perulangan `while` untuk mencari simpul terakhir dalam daftar (simpul yang penunjuk `next`-nya bernilai `null`). Setelah simpul terakhir ditemukan, alurnya dilanjutkan dengan mengubah penunjuk `next` pada simpul tersebut agar mengarah ke `newNode`.

```
16 // simpan head ke variabel sementara
17 NodeSLL_2511533025 last = head_3025;
18 // telusuri ke node akhir
19 while (last.next_3025 != null) {
20 last = last.next_3025;
21 }
22 // ubah pointer
23 last.next_3025 = newNode;
24 return head_3025;
25 }
```

4. Kode fungsi statis bernama `GetNode` yang berfungsi sebagai factory method untuk membuat objek baru dari kelas `NodeSLL_2511533025`. Fungsi ini menerima satu parameter input berupa bilangan bulat (`int data`), yang kemudian diteruskan ke dalam constructor saat menginisialisasi node baru menggunakan kata kunci `new`.

```
26 static NodeSLL_2511533025 GetNode(int data) {
27 return new NodeSLL_2511533025(data);
28 }
29 }
```

5. Kode ini untuk menyisipkan sebuah simpul (node) baru ke dalam struktur data Single Linked List berdasarkan posisi tertentu. Di awal fungsi, kode memvalidasi input posisi; jika posisi kurang dari satu, sistem akan mencetak pesan kesalahan

"Invalid position". Jika posisi yang diminta adalah satu (penyisipan di awal daftar), kode akan membuat objek `new_node` baru dengan nilai yang diberikan, mengarahkan penunjuk `next` dari simpul baru tersebut ke `head` yang lama agar daftar tetap terhubung, lalu mengembalikan simpul baru tersebut sebagai `head` yang baru bagi daftar tersebut.

```

30 static NodeSLL_2511533025 insertPos(NodeSLL_2511533025 headNode_3025, int position_3025, int value_3025) {
31     NodeSLL_2511533025 head = headNode_3025;
32     if (position_3025 < 1)
33         System.out.print("Invalid position");
34     if (position_3025 == 1) {
35         NodeSLL_2511533025 new_node = new NodeSLL_2511533025(value_3025);
36         new_node.next_3025 = head;
37         return new_node;

```

6. Kode ini berfungsi untuk menyisipkan sebuah simpul (node) baru ke dalam struktur data Single Linked List pada posisi tertentu selain di awal. Program menggunakan perulangan `while` untuk menelusuri daftar sambil mengurangi nilai `position_3025` hingga mencapai posisi yang diinginkan (saat bernilai 1). Jika posisi ditemukan, sebuah simpul baru dibuat menggunakan fungsi `GetNode`, lalu dilakukan pemutusan dan penyambungan tautan (pointer `next`) agar simpul baru berada di antara simpul saat ini dan simpul berikutnya, diikuti dengan perintah `break` untuk menghentikan iterasi.

```

38     } else {
39         while (position_3025-- != 0) {
40             if (position_3025 == 1) {
41                 NodeSLL_2511533025 newNode = GetNode(value_3025);
42                 newNode.next_3025 = headNode_3025.next_3025;
43                 headNode_3025.next_3025 = newNode;
44                 break;
45             }
46             headNode_3025 = headNode_3025.next_3025;
47         }
48         if (position_3025 != 1)
49             System.out.print("Posisi di luar jangkauan");
50     }
51     return head;
52 }

```

7. Kode ini mendefinisikan sebuah metode bernama `printList` yang berfungsi untuk menampilkan seluruh elemen data dalam struktur data Single Linked List ke layar secara berurutan. Proses dimulai dengan menetapkan variabel bantu `curr` pada simpul pertama (`head`), kemudian menggunakan perulangan `while` untuk menelusuri list dan mencetak nilai data setiap simpul diikuti tanda panah (`-->`) selama simpul berikutnya tidak kosong. Setelah mencapai simpul terakhir, perulangan berhenti dan program mencetak data simpul.

```

53 public static void printList(NodeSLL_2511533025 head) {
54     NodeSLL_2511533025 curr = head;
55     while (curr.next_3025 != null) {
56         System.out.print(curr.data_3025+"-->");
57         curr = curr.next_3025;
58     }
59     if (curr.next_3025==null) {
60         System.out.print(curr.data_3025);
61     }
62     System.out.println();
63 }
64

```

8. Kode ini untuk menginisialisasi atau membangun sebuah struktur data Single Linked List secara manual. Proses ini dimulai dengan membuat objek node pertama sebagai head yang menyimpan nilai 2, kemudian secara bertahap menghubungkan node-node baru ke atribut next_3025 dari node sebelumnya untuk memasukkan nilai 3, 5, dan 6 secara berurutan. Hasil akhirnya adalah sebuah rantai linear data di mana setiap elemen (node) menyimpan referensi ke elemen berikutnya, membentuk urutan list 2 -> 3 -> 5 -> 6.

```

65 public static void main(String[] args) {
66     // buat linked list 2->3->5->6
67     NodeSLL_2511533025 head_3025 = new NodeSLL_2511533025(2);
68     head_3025.next_3025 = new NodeSLL_2511533025(3);
69     head_3025.next_3025.next_3025 = new NodeSLL_2511533025(5);
70     head_3025.next_3025.next_3025.next_3025 = new NodeSLL_2511533025(6);
71

```

9. Kode ini mendemonstrasikan berbagai operasi manipulasi pada struktur data linked list yang dikelola melalui variabel head_3025. Alurnya dimulai dengan mencetak isi senarai awal, kemudian secara berturut-turut melakukan penyisipan simpul baru: pertama menambahkan data bernilai 1 di bagian depan menggunakan fungsi insertAtFront, diikuti penambahan data bernilai 7 di bagian belakang dengan insertAtEnd, dan terakhir menyisipkan data bernilai 4 pada posisi spesifik (posisi ke-4) menggunakan fungsi insertPos.

```

71 // cetak list asli
72 System.out.print("Senarai berantai awal:");
73 printList(head_3025);
74 // tambahkan node baru di depan
75 System.out.print("tambah 1 simpul di depan: ");
76 int data = 1;
77 head_3025 = insertAtFront(head_3025, data);
78 // cetak update list
79 printList(head_3025);
80 // tambahkan node baru di belakang
81 System.out.print("tambah 1 simpul di belakang: ");
82 int data2 = 7;
83 head_3025 = insertAtEnd(head_3025, data2);
84 // cetak update list
85 printList(head_3025);
86 System.out.print("tambah 1 simpul ke data 4: ");
87 int data3 = 4;
88 int pos=4;
89 head_3025 = insertPos(head_3025,pos,data3);
90 // cetak update list
91 printList(head_3025);
92 }
93

```

c. Class PencarianSLL_2511533025

1. Kode ini untuk mendefinisikan sebuah metode statis bernama `searchKey` yang berfungsi untuk mencari keberadaan nilai tertentu (`key_3025`) di dalam sebuah struktur data Single Linked List. Proses pencarian dimulai dengan menginisialisasi penunjuk `curr_3025` pada bagian kepala (`head`) list, kemudian menggunakan perulangan `while` untuk menelusuri setiap simpul (`node`) selama penunjuk tersebut tidak bernilai `null`. Di dalam setiap iterasi, kode akan membandingkan data pada simpul saat ini dengan kunci yang dicari, jika ditemukan kecocokan, metode akan langsung mengembalikan nilai `true`. Jika seluruh list telah ditelusuri hingga akhir dan kunci tidak ditemukan, maka penelusuri akan keluar dari perulangan dan mengembalikan nilai `false`.

```
DeklarasiVar... OperatorAri... OperatorAssi... NodeSLL_251... TambahSLL_25... Per
1 package Pekan5_2511533025;
2 public class PencarianSLL_2511533025 {
3     static boolean searchKey(NodeSLL_2511533025 head_3025, int key_3025) {
4         NodeSLL_2511533025 curr_3025 = head_3025;
5         while (curr_3025 != null) {
6             if (curr_3025.data_3025 == key_3025)
7                 return true;
8             curr_3025 = curr_3025.next_3025;
9         }
10        return false;
11    }
12 }
```

2. Kode ini untuk implementasi metode traversal pada struktur data Single Linked List dalam bahasa pemrograman Java, yang berfungsi untuk mengunjungi dan menampilkan seluruh data yang tersimpan di dalam list secara berurutan. Proses dimulai dengan menginisialisasi variabel bantu `curr_3025` yang menunjuk ke bagian `head` (elemen pertama), kemudian menggunakan perulangan `while` untuk menelusuri setiap simpul (`node`) selama variabel tersebut tidak bernilai `null`.

```
13 public static void traversal(NodeSLL_2511533025 head_3025) {
14     //mulai dari head
15     NodeSLL_2511533025 curr_3025 = head_3025;
16     //telusuri sampai pointer null
17     while (curr_3025 != null) {
18         System.out.print(" " + curr_3025.data_3025);
19         curr_3025 = curr_3025.next_3025;
20     }
21     System.out.println();
22 }
23 }
```

3. Kode ini untuk membangun struktur data Single Linked List secara manual dan melakukan penelusuran (`traversal`) terhadap isi list tersebut. Di dalam metode `main`, sebuah objek `head` dibuat sebagai simpul pertama dengan nilai 14, yang kemudian diikuti dengan penambahan empat simpul baru (bernilai 21, 13, 30, dan 10) secara berantai menggunakan atribut `next_3025`. Setelah daftar tersebut terbentuk dengan total lima simpul,

program mencetak teks "Penelusuran SLL : " dan memanggil fungsi traversal(head) untuk mengunjungi serta menampilkan nilai dari setiap simpul secara berurutan mulai dari kepala list hingga elemen terakhir.

```
24 public static void main(String[] args) {
25     NodeSLL_2511533025 head = new NodeSLL_2511533025(14);
26     head.next_3025 = new NodeSLL_2511533025(21);
27     head.next_3025.next_3025 = new NodeSLL_2511533025(13);
28     head.next_3025.next_3025.next_3025 = new NodeSLL_2511533025(30);
29     head.next_3025.next_3025.next_3025.next_3025 = new NodeSLL_2511533025(10);
30
31     System.out.print("Penelusuran SLL : ");
32     traversal(head);
33 }
```

4. Kode ini untuk melakukan pencarian data tertentu di dalam sebuah struktur data (kemungkinan linked list berdasarkan parameter head) dengan nilai kunci (key) sebesar 30. Jika fungsi mengembalikan nilai benar (true), program akan menampilkan pesan "ketemu", dan jika tidak ditemukan, program akan menampilkan pesan "tidak ada".

```
34 // data yang akan dicari
35 int key = 30;
36 System.out.print("cari data " + key + " = ");
37 if (searchKey(head, key))
38     System.out.println("ketemu");
39 else
40     System.out.println("tidak ada");
41 }
42 }
```

d. Class HapusSLL_2511533025

1. Kode fungsi bernama deleteHead yang bertujuan untuk menghapus simpul pertama (head) dari sebuah Single Linked List (SLL). Fungsi ini bekerja dengan cara memeriksa terlebih dahulu apakah list tersebut kosong (head == null), di mana jika kosong maka akan langsung mengembalikan nilai null. Jika list berisi data, proses penghapusan dilakukan secara logis dengan memindahkan penunjuk head ke simpul berikutnya melalui perintah head = head.next_3025. Terakhir, fungsi ini mengembalikan referensi head yang baru, yang kini menunjuk ke simpul kedua dari list aslinya atau menjadi null.

```
DeklarasiVar... OperatorAri... OperatorAssi... NodeSLL_251... TambahSLL_25... PencarianS
1 package Pekan5_2511533025;
2 public class HapusSLL_2511533025 {
3     // fungsi untuk menghapus head
4     public static NodeSLL_2511533025 deleteHead(NodeSLL_2511533025 head) {
5         // jika SLL kosong
6         if (head == null)
7             return null;
8         // pindahkan head ke node berikutnya
9         head = head.next_3025;
10        // Return head baru
11        return head;
12    }
}
```

2. Kode fungsi `removeLastNode` untuk menghapus simpul (node) terakhir dalam struktur data Singly Linked List. Fungsi ini pertama-tama melakukan pengecekan kondisi dasar, jika daftar kosong (`head == null`), ia langsung mengembalikan `null`, dan jika daftar hanya berisi satu simpul (`head.next_3025 == null`), simpul tersebut dianggap terhapus dengan mengembalikan `null`. Secara keseluruhan, logika ini bertujuan untuk memutus hubungan ke simpul terakhir dengan mengidentifikasi simpul kedua dari belakang dan mengatur penunjuk (pointer) berikutnya menjadi `null`.

```
13 // fungsi menghapus node terakhir SLL
14 public static NodeSLL_2511533025 removeLastNode(NodeSLL_2511533025 head) {
15     // jika list kosong, return null
16     if (head == null) {
17         return null;
18     }
19     // jika list satu node, hapus node dan return null
20     if (head.next_3025 == null) {
21         return null;
22     }
```

3. Kode ini untuk menghapus node terakhir dalam sebuah Single Linked List. Proses dimulai dengan menginisialisasi variabel `secondLast` pada bagian `head`, kemudian menggunakan perulangan `while` untuk menelusuri list hingga menemukan node yang berada tepat sebelum node terakhir; hal ini dideteksi dengan memeriksa apakah `next.next` dari node saat ini bernilai `null`. Setelah posisi node kedua terakhir ditemukan, kode akan memutus hubungan ke node terakhir dengan mengubah referensi `next` pada node tersebut menjadi `null`

```
23 //temukan node terakhir ke dua
24 NodeSLL_2511533025 secondLast = head;
25 while (secondLast.next_3025.next_3025 != null) {
26     secondLast = secondLast.next_3025;
27 }
28 // hapus node terakhir
29 secondLast.next_3025 = null;
30 return head;
31 }
```

4. Kode ini mengimplementasikan fungsi `deleteNode` untuk menghapus sebuah node pada posisi tertentu dalam sebuah Single Linked List. Fungsi ini bekerja dengan terlebih dahulu mengecek apakah daftar kosong atau jika node yang akan dihapus berada di posisi pertama (kepala/head), di mana ia akan langsung memperbarui `head` ke node berikutnya. Jika posisi yang diminta berada di tengah atau akhir, kode menggunakan perulangan `for` untuk menelusuri list hingga mencapai posisi yang ditentukan sambil menyimpan referensi ke node sebelumnya (`prev_3025`). Setelah posisi ditemukan dan dipastikan node tersebut tidak `null`, penghapusan dilakukan dengan menghubungkan `next` dari node sebelumnya langsung ke node setelah node yang dihapus, secara efektif melewati dan memutus node target dari struktur list.

```

32 // fungsi menghapus node di posisi tertentu
33 public static NodeSLL_2511533025 deleteNode(NodeSLL_2511533025 head, int position) {
34     NodeSLL_2511533025 temp_3025 = head;
35     NodeSLL_2511533025 prev_3025 = null;
36     // jika linked list null
37     if (temp_3025 == null)
38         return head;
39     // kasus 1: head dihapus
40     if (position == 1) {
41         head = temp_3025.next_3025;
42         return head;
43     }
44     // kasus 2: menghapus node di tengah
45     // telusuri ke node yang dihapus
46     for (int i = 1; temp_3025 != null && i < position; i++) {
47         prev_3025 = temp_3025;
48         temp_3025 = temp_3025.next_3025;
49     }
50     // jika ditemukan, hapus node
51     if (temp_3025 != null) {
52         prev_3025.next_3025 = temp_3025.next_3025;
53     } else {
54         System.out.println("Data tidak ada");
55     }
56     return head;
57 }

```

- Kode ini mendefinisikan sebuah fungsi Java bernama printList yang bertujuan untuk menampilkan semua elemen data dalam sebuah Single Linked List (SLL) ke layar dalam satu baris. Fungsi ini bekerja dengan cara menerima parameter berupa kepala list (head), lalu melakukan iterasi menggunakan perulangan while untuk menelusuri setiap simpul selama simpul berikutnya tidak bernilai null. Di dalam perulangan tersebut, data dari simpul saat ini dicetak diikuti dengan simbol panah ("->") untuk menunjukkan hubungan antar simpul, kemudian penunjuk berpindah ke simpul selanjutnya.

```

54 // fungsi mencetak SLL
55 public static void printList(NodeSLL_2511533025 head) {
56     NodeSLL_2511533025 curr_3025 = head;
57     while (curr_3025.next_3025 != null) {
58         System.out.print(curr_3025.data_3025+"->");
59         curr_3025 = curr_3025.next_3025;
60     }
61     if (curr_3025.next_3025 == null) {
62         System.out.print(curr_3025.data_3025);
63     }
64     System.out.println();
65 }

```

- Kode ini merupakan implementasi pembuatan struktur data Single Linked List (SLL) secara manual di dalam metode main menggunakan bahasa pemrograman Java. Baris-baris kode ini berfungsi untuk menginisialisasi enam buah objek node dari kelas NodeSLL_2511533025, di mana node pertama ditetapkan sebagai head dengan nilai 1, lalu setiap node berikutnya dihubungkan secara berantai melalui atribut next_3025. Proses ini membentuk urutan linear data dari angka 1 hingga 6 yang berakhir pada null.

```

63 // kelas main
64 public static void main(String[] args) {
65     // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
66     NodeSLL_2511533025 head = new NodeSLL_2511533025(1);
67     head.next_3025 = new NodeSLL_2511533025(2);
68     head.next_3025.next_3025 = new NodeSLL_2511533025(3);
69     head.next_3025.next_3025.next_3025 = new NodeSLL_2511533025(4);
70     head.next_3025.next_3025.next_3025.next_3025 = new NodeSLL_2511533025(5);
71     head.next_3025.next_3025.next_3025.next_3025.next_3025 = new NodeSLL_2511533025(6);
72 }

```

- Kode ini merupakan implementasi operasi manipulasi pada struktur data linked list dalam bahasa pemrograman Java yang

bertujuan untuk menunjukkan cara menghapus simpul (node) dari berbagai posisi. Secara berurutan, program ini mencetak daftar awal, menghapus elemen pertama (head) menggunakan fungsi `deleteHead`, menghapus elemen terakhir dengan `removeLastNode`, serta menghapus elemen pada indeks tertentu (posisi 2) melalui fungsi `deleteNode`. Di setiap tahapnya, variabel `head` diperbarui untuk menjaga integritas struktur daftar.

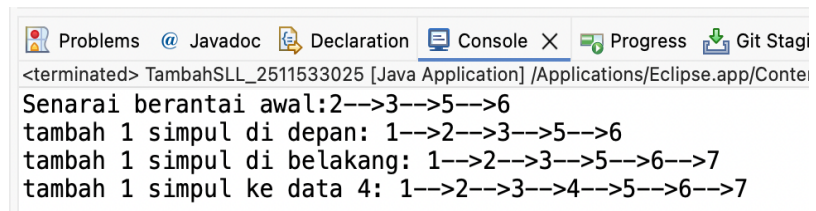
```

72 // cetak list awal
73 System.out.println("list awal: ");
74 printList(head);
75 // hapus head
76 head = deleteHead(head);
77 System.out.println("List setelah head dihapus: ");
78 printList(head);
79 //hapus node terakhir
80 head = removeLastNode(head);
81 System.out.println("List setelah simpul terakhir di hapus: ");
82 printList(head);
83 // Deleting node at position 2
84 int position = 2;
85 head = deleteNode(head, position);
86 // Print list after deletion
87 System.out.println("List setelah posisi 2 dihapus: ");
88 printList(head);
89 }
90
91 }

```

f. Hasil projek pekan 4.

1. Output dari Class `NodeSLL_2511533025` dan Class `TambahSLL_2511533025`.

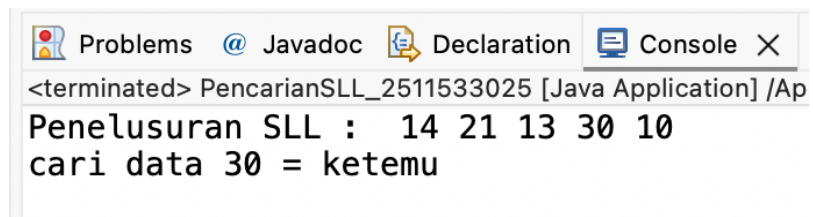


```

<terminated> TambahSLL_2511533025 [Java Application] /Applications/Eclipse.app/Conte
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7

```

2. Output dari Class `PencarianSLL_2511533025`.

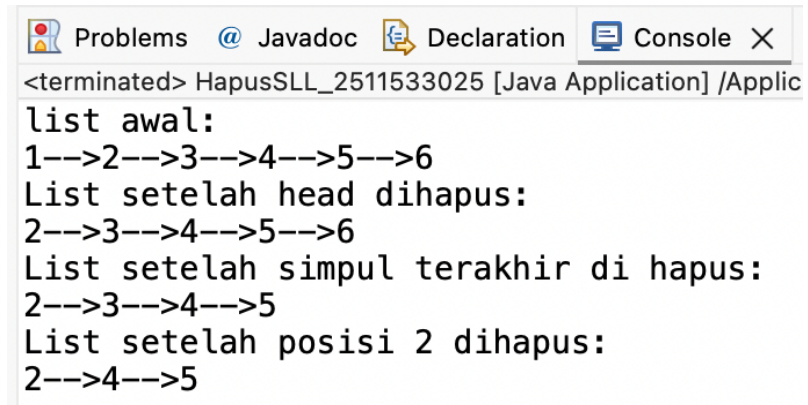


```

<terminated> PencarianSLL_2511533025 [Java Application] /Ap
Penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu

```

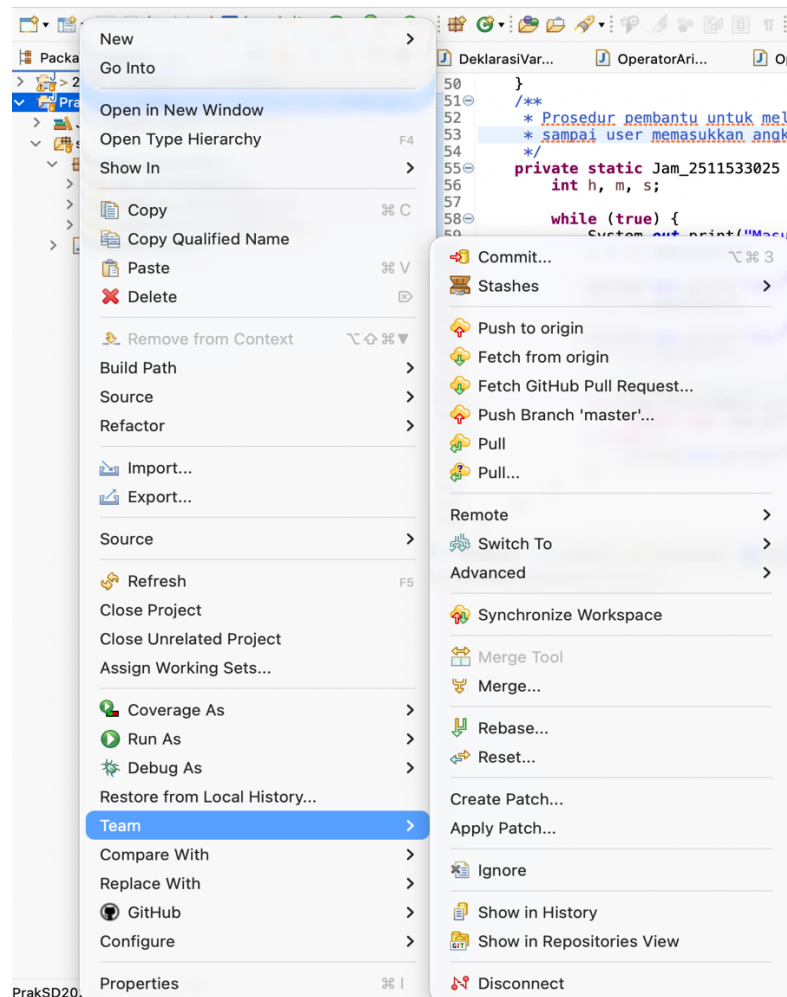
3. Output dari Class `HapusSLL_2511533025`.



```
<terminated> HapusSLL_2511533025 [Java Application] /Applic
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

B. Langkah Penyimpanan

1. Sebelum kita masuk ke langkah, kita harus buat akun github dulu. Jika sudah buat akun github, baru bisa kita simpan di githubnya. Selanjutnya ikutin langkah ini. Tekan kanan mouse pada folder proyek, setelah itu pilih team, terus ke *commit*.



2. Lalu tekan tombol plus 2/double plus warna hijau jika ingin push semua kode, kalau hanya salah satu klik kode yang ingin di push, terus klik tanda plus single.

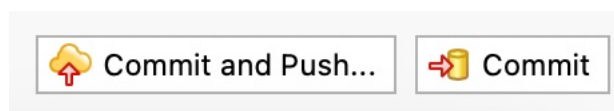


3. Lalu masukan pesan pada kolom *commit message*.

Commit Message

i Unborn branch: this commit will create the branch 'master'.

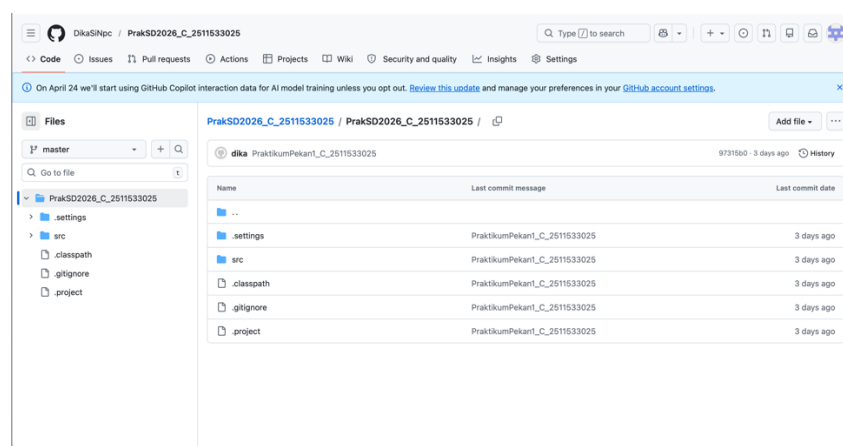
4. Setelah itu tekan *commit* and pus, lalu ikutin arahan untuk memasukan nama dan pw akun git hub kalian, maka proyek kalian sudah tersimpan di akun github kalian.



5. Proyek yang sudah disimpan di akun proyek.



6. Gambar proyek apabila sudah masuk di akun github.



Sedikit tambahan jika ingin cek hasil program nya tekan tombol *run* warna hijau.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru atau package yang baru.

BAB 3

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil Praktikum Struktur Data tentang *Single Linked List* memberikan pemahaman bahwa struktur data ini sangat efektif untuk mengelola data secara dinamis karena setiap elemen (*node*) dapat ditambahkan atau dihapus tanpa perlu menggeser elemen lain seperti pada *array*. Melalui praktikum, penulis berhasil mempelajari konsep *pointer* untuk menghubungkan antar node, serta mengimplementasikan operasi dasar seperti *insert head*, *insert tail*, *insert at position*, *delete*, *search*, dan *traversing*. Hal ini membuktikan bahwa *Single Linked List* lebih fleksibel dalam alokasi memori dan efisien untuk operasi penyisipan atau penghapusan di awal atau akhir list, meskipun memiliki keterbatasan dalam akses data yang harus dilakukan secara sekuensial dari head.

3.2 Saran

Untuk pengembangan selanjutnya, disarankan agar mahasiswa juga mempelajari variasi *Linked List* lainnya seperti *Double Linked List* untuk memahami kelebihan dan kekurangan masing-masing struktur. Mahasiswa juga sebaiknya mencoba mengimplementasikan *Single Linked List* dalam studi kasus nyata, seperti manajemen antrian atau daftar kontak, agar lebih memahami aplikasi praktisnya. Terakhir, pelajari juga kompleksitas waktu dari setiap operasi *Linked List* agar dapat memilih struktur data yang paling optimal sesuai kebutuhan program.

DAFTAR PUSTAKA

- Sumber daring (website):

[1] Teknik Elektro dan Informatika UM, "Struktur Data - Modul Praktikum 6: Single Linked List," TEI Universitas Negeri Malang. <https://tei.um.ac.id/wp-content/uploads/2016/04/Struktur-Data-Modul-Praktikum-6-Single-Linked-List.pdf> [Diakses: 12-Sep-2025].

[2] Universitas Pembangunan Jaya, "Handout TIF104 - Modul Praktikum Struktur Data," OCW UPJ. <https://ocw.upj.ac.id/files/Handout-TIF104-Modul-Praltikum-Struktur-Data.pdf> [Diakses: 12-Sep-2025].

[3] S. Frieda, "Praktikum Struktur Data: Linked List," Medium. <https://medium.com/@saskefrieda/praktikum-struktur-data-linked-list-6e820fc0afde> [Diakses: 12-Sep-2025].

LAPORAN TUGAS PRAKTIKUM PEKAN 5

1. Intruksi Tugas Praktikum Pekan 5

Topik : Implementasi Single Linked List

Tema : Simulasi Antrian Rumah Sakit

Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien

yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO/First In, First Out).

Aturan Penamaan (Wajib Dipatuhi):

- Nama Kelas : Gunakan NIM lengkap. Contoh : Pasien_2511533025 dan kelas ke 2 RumahSakit_2411531005.
- Nama Variabel dan Method : Setiap variabel dan method wajib menggunakan 4 digit terakhir NIM. Contoh : String namaPasien_3025, String penyakit_3025.
- Pointer next : Variabel pointer ke node berikutnya juga wajib diberi akhiran 4 digit NIM. Contoh : Pasien_2511533025 next_3025.

Struktur Program:

1. Kelas ADT Pasien (Pasien_NIMLengkap)

Atribut :

- Nama Pasien
- Keluhan / Penyakit
- Nomor Antrian (int)
- Pointer next ke node Pasien berikutnya.

Method :

- Constructor untuk menginisialisasi semua atribut.
- Selektor (Getter) untuk setiap atribut.
- Mutator (Setter) untuk setiap atribut termasuk pointer next.

2. Kelas Driver (RumahSakit_NIMLengkap):

1. Daftarkan Pasien (Insert at Tail) : Menambahkan node pasien baru di akhir linked list. Jika list kosong, node baru langsung menjadi head. Gunakan sebuah variabel counter untuk menyimpan nomor antrian terakhir. Setiap kali fungsi Daftarkan Pasien dipanggil, nomor antrian akan bertambah secara otomatis (auto-increment).

2. Panggil Pasien (Delete Head) : Menghapus node terdepan dan menampilkan data pasien yang dipanggil. Head digeser ke node berikutnya.
3. Tampilkan Antrian (Display) : Menelusuri linked list dari head hingga null dan menampilkan semua data pasien beserta posisinya.
4. Cari Pasien (Search) : Pencarian dilakukan berdasarkan nama pasien secara CaseInsensitive (tidak membedakan huruf besar dan kecil). Tampilkan pesan jika tidak ditemukan.
5. Cek Status Antrian : Menampilkan jumlah total pasien dan informasi pasien terdepan.
Tampilkan pesan jika list kosong.

2. Kode program Class Pasien_2511533025

```
package TugasPekan5_2511533025;
public class Pasien_2511533025 {
    private String namaPasien_3025;
    private String penyakit_3025;
    private int nomorAntrian_3025;
    private Pasien_2511533025 next_3025;
    // Constructor
    public Pasien_2511533025(String nama_3025, String
penyakit_3025, int nomor_3025) {
        this.namaPasien_3025 = nama_3025;
        this.penyakit_3025 = penyakit_3025;
        this.nomorAntrian_3025 = nomor_3025;
        this.next_3025 = null;
    }
    // Getter
    public String getNamaPasien_3025() {
        return namaPasien_3025;
    }
    public String getPenyakit_3025() {
        return penyakit_3025;
    }
    public int getNomorAntrian_3025() {
        return nomorAntrian_3025;
    }
    public Pasien_2511533025 getNext_3025() {
        return next_3025;
    }
    // Setter
    public void setNamaPasien_3025(String nama_3025)
{
        this.namaPasien_3025 = nama_3025;
    }
    public void setPenyakit_3025(String
penyakit_3025) {
        this.penyakit_3025 = penyakit_3025;
    }
}
```

```

        public void setNomorAntrian_3025(int nomor_3025)
        {
            this.nomorAntrian_3025 = nomor_3025;
        }
        public void setNext_3025(Pasien_2511533025
next_3025) {
            this.next_3025 = next_3025;
        }
    }

```

3. Kode program Class DriverRumahSakit_2511533025

```

package TugasPekan5_2511533025;
import java.util.Scanner;

public class DriverRumahSakit_2511533025 {
    private Pasien_2511533025 head_3025;
    private int counter_3025 = 0;
    // Method Daftarkan Pasien (Insert)
    public void daftarkanPasien_3025(String nama_3025,
String penyakit_3025) {
        counter_3025++;
        Pasien_2511533025 baru_3025 =
            new Pasien_2511533025(nama_3025,
penyakit_3025, counter_3025);

        if (head_3025 == null) {
            head_3025 = baru_3025;
        } else {
            Pasien_2511533025 temp_3025 = head_3025;
            while (temp_3025.getNext_3025() != null)
            {
                temp_3025 = temp_3025.getNext_3025();
            }
            temp_3025.setNext_3025(baru_3025);
        }

        System.out.println("Pasien berhasil
didaftarkan! Nomor Antrian: " + counter_3025);
    }
    // Method Panggil Pasien (Delete Head)
    public void panggilPasien_3025() {
        if (head_3025 == null) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Pasien dipanggil:");
            System.out.println("Nama : " +
head_3025.getNamaPasien_3025());
            System.out.println("Keluhan : " +
head_3025.getPenyakit_3025());
            System.out.println("Nomor Antrian : " +
head_3025.getNomorAntrian_3025());
        }
    }
}

```

```

        head_3025 = head_3025.getNext_3025();
    }
}
// Method Tampilkan Antrian (Display)
public void tampilkanAntrian_3025() {
    if (head_3025 == null) {
        System.out.println("Antrian kosong!");
        return;
    }

    Pasien_2511533025 temp_3025 = head_3025;
    while (temp_3025 != null) {
        System.out.println("No: " +
temp_3025.getNomorAntrian_3025() +
        " | Nama: " +
temp_3025.getNamaPasien_3025() +
        " | Keluhan: " +
temp_3025.getPenyakit_3025());
        temp_3025 = temp_3025.getNext_3025();
    }
}
// Method Cari Pasien (Search)
public void cariPasien_3025(String nama_3025) {
    Pasien_2511533025 temp_3025 = head_3025;

    while (temp_3025 != null) {
        if
(temp_3025.getNamaPasien_3025().equalsIgnoreCase(nama
_3025)) {
            System.out.println("Pasien
ditemukan!");
            System.out.println("No Antrian: " +
temp_3025.getNomorAntrian_3025());
            return;
        }
        temp_3025 = temp_3025.getNext_3025();
    }

    System.out.println("Pasien tidak
ditemukan!");
}
// Method Cek Status Antrian
public void cekStatus_3025() {
    if (head_3025 == null) {
        System.out.println("Antrian kosong!");
        return;
    }

    int jumlah_3025 = 0;
    Pasien_2511533025 temp_3025 = head_3025;

    while (temp_3025 != null) {
        jumlah_3025++;
        temp_3025 = temp_3025.getNext_3025();
    }
}

```

```

    }

    System.out.println("Jumlah pasien: " +
jumlah_3025);
    System.out.println("Pasien terdepan: " +
head_3025.getNamaPasien_3025());
}
// Method Tampilkan Menu dan Main Program
public static void main(String[] args) {
    Scanner input_3025 = new Scanner(System.in);
    DriverRumahSakit_2511533025 rs_3025 = new
DriverRumahSakit_2511533025();

    int pilih_3025;

    do {
        System.out.println("\n=== Antrian Rumah
Sakit NIM: 2511533025 ===");
        System.out.println("1. Daftarkan Pasien
(Insert)");
        System.out.println("2. Panggil Pasien
(Delete Head)");
        System.out.println("3. Tampilkan Antrian
(Display)");
        System.out.println("4. Cari Pasien
(Search)");
        System.out.println("5. Cek Status
Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");

        pilih_3025 = input_3025.nextInt();
        input_3025.nextLine();

        switch (pilih_3025) {
            case 1:
                System.out.print("Masukkan Nama
Pasien : ");
                String nama_3025 =
input_3025.nextLine();
                System.out.print("Masukkan
Keluhan : ");
                String keluhan_3025 =
input_3025.nextLine();
                rs_3025.daftarPasien_3025(nama_3025, keluhan_3025);
                break;

            case 2:
                rs_3025.panggilPasien_3025();
                break;

            case 3:
                rs_3025.tampilkanAntrian_3025();

```

```

        break;

    case 4:
        System.out.print("Masukkan Nama : ");
        String cari_3025 = input_3025.nextLine();
        rs_3025.cariPasien_3025(cari_3025);
        break;

    case 5:
        rs_3025.cekStatus_3025();
        break;

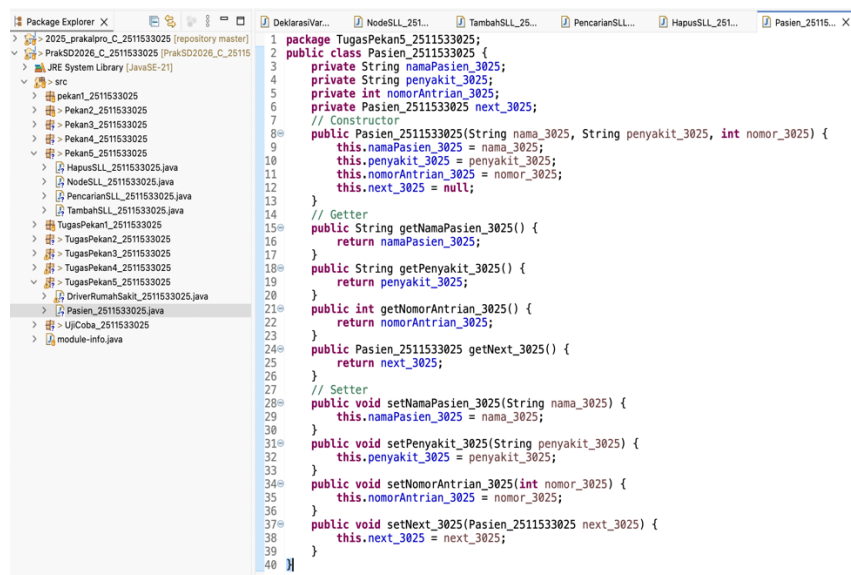
    case 6:
        System.out.println("Terima
        kasih.");
        break;

    default:
        System.out.println("Pilihan tidak
        valid!");
    }
} while (pilih_3025 != 6);
}
}

```

4. Gambar kode program dan output program.

1. Gambar Class Pasien_2511533025



2. Gambar Class DriverRumahSakit_2511533025



```
1 package TugasPekan5_2511533025;
2 import java.util.Scanner;
3
4 public class DriverRumahSakit_2511533025 {
5     private Pasien_2511533025 head_3025;
6     private int counter_3025 = 0;
7     // Method Daftarkan Pasien (Insert)
8     public void daftarPasien_3025(String nama_3025, String penyakit_3025) {
9         counter_3025++;
10        Pasien_2511533025 baru_3025 =
11            new Pasien_2511533025(nama_3025, penyakit_3025, counter_3025);
12
13        if (head_3025 == null) {
14            head_3025 = baru_3025;
15        }
16        else {
17            Pasien_2511533025 temp_3025 = head_3025;
18            while (temp_3025.getNext_3025() != null) {
19                temp_3025 = temp_3025.getNext_3025();
20            }
21            temp_3025.setNext_3025(baru_3025);
22        }
23
24        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_3025);
25    }
26    // Method Panggil Pasien (Delete Head)
27    public void panggilPasien_3025() {
28        if (head_3025 == null) {
29            System.out.println("Antrian kosong!");
30        }
31        else {
32            System.out.println("Pasien dipanggil!");
33            System.out.println("Nama : " + head_3025.getNamaPasien_3025());
34            System.out.println("Keluhan : " + head_3025.getPenyakit_3025());
35            System.out.println("Nomor Antrian : " + head_3025.getNomorAntrian_3025());
36            head_3025 = head_3025.getNext_3025();
37        }
38    }
39    // Method Tampilkan Antrian (Display)
40    public void tampilkanAntrian_3025() {
41        if (head_3025 == null) {
42            System.out.println("Antrian kosong!");
43            return;
44        }
45        Pasien_2511533025 temp_3025 = head_3025;
46        while (temp_3025 != null) {
47            System.out.println("No: " + temp_3025.getNomorAntrian_3025() +
48                " | Nama: " + temp_3025.getNamaPasien_3025() +
49                " | Keluhan: " + temp_3025.getPenyakit_3025());
50            temp_3025 = temp_3025.getNext_3025();
51        }
52    }
53    // Method Cari Pasien (Search)
54    public void cariPasien_3025(String nama_3025) {
55        Pasien_2511533025 temp_3025 = head_3025;
56        while (temp_3025 != null) {
57            if (temp_3025.getNamaPasien_3025().equalsIgnoreCase(nama_3025)) {
58                System.out.println("Pasien ditemukan!");
59                System.out.println("No Antrian: " + temp_3025.getNomorAntrian_3025());
60                return;
61            }
62            temp_3025 = temp_3025.getNext_3025();
63        }
64        System.out.println("Pasien tidak ditemukan!");
65    }
66 }
67
```

```

> 2025_prakalpro_C_2511533025 [repository master]
> PrakSD2026_C_2511533025 [PrakSD2026_C_2511533025]
> src
  > pekan1_2511533025
  > pekan2_2511533025
  > pekan3_2511533025
  > pekan4_2511533025
  > pekan5_2511533025
  > pekan6_2511533025
  > HapusSL_2511533025.java
  > HapusSL_2511533025.java
  > PencarianSL_2511533025.java
  > PencarianSL_2511533025.java
  > TambahSL_2511533025.java
  > TambahSL_2511533025.java
  > TugasPekan1_2511533025
  > TugasPekan2_2511533025
  > TugasPekan3_2511533025
  > TugasPekan4_2511533025
  > TugasPekan5_2511533025
  > TugasPekan6_2511533025
  > DriverRumahSakit_2511533025.java
  > Pasien_2511533025.java
  > UjiCoba_2511533025
  > module-info.java

  > src
    > pekan1_2511533025
    > pekan2_2511533025
    > pekan3_2511533025
    > pekan4_2511533025
    > pekan5_2511533025
    > pekan6_2511533025
    > HapusSL_2511533025.java
    > HapusSL_2511533025.java
    > PencarianSL_2511533025.java
    > PencarianSL_2511533025.java
    > TambahSL_2511533025.java
    > TambahSL_2511533025.java
    > TugasPekan1_2511533025
    > TugasPekan2_2511533025
    > TugasPekan3_2511533025
    > TugasPekan4_2511533025
    > TugasPekan5_2511533025
    > TugasPekan6_2511533025
    > DriverRumahSakit_2511533025.java
    > Pasien_2511533025.java
    > UjiCoba_2511533025
    > module-info.java

68 // Method Cek Status Antrian
69 public void cekStatus_3025() {
70     if (head_3025 == null) {
71         System.out.println("Antrian kosong!");
72         return;
73     }
74     int jumlah_3025 = 0;
75     Pasien_2511533025 temp_3025 = head_3025;
76     while (temp_3025 != null) {
77         jumlah_3025++;
78         temp_3025 = temp_3025.getNext_3025();
79     }
80     System.out.println("Jumlah pasien: " + jumlah_3025);
81     System.out.println("Pasien terdepan: " + head_3025.getNamaPasien_3025());
82 }
83 // Method Tampilkan Menu dan Main Program
84 public static void main(String[] args) {
85     Scanner input_3025 = new Scanner(System.in);
86     DriverRumahSakit_2511533025 rs_3025 = new DriverRumahSakit_2511533025();
87     int pilih_3025;
88     do {
89         System.out.println("===== Antrian Rumah Sakit NIM: 2511533025 =====");
90         System.out.println("1. Daftarkan Pasien (Insert)");
91         System.out.println("2. Panggil Pasien (Delete Head)");
92         System.out.println("3. Tampilkan Antrian (Display)");
93         System.out.println("4. Cari Pasien (Search)");
94         System.out.println("5. Cek Status Antrian");
95         System.out.println("6. Keluar");
96         System.out.print("Pilihan: ");
97         pilih_3025 = input_3025.nextInt();
98         input_3025.nextLine();
99         switch (pilih_3025) {
100             case 1:
101                 System.out.print("Masukkan Nama Pasien : ");
102                 String nama_3025 = input_3025.nextLine();
103                 System.out.print("Masukkan Keluhan : ");
104                 String keluhan_3025 = input_3025.nextLine();
105                 rs_3025.daftarPasien_3025(nama_3025, keluhan_3025);
106                 break;
107             case 2:
108                 rs_3025.panggilPasien_3025();
109                 break;
110             case 3:
111                 rs_3025.tampilkanAntrian_3025();
112                 break;
113             case 4:
114                 System.out.print("Masukkan Nama : ");
115                 String cari_3025 = input_3025.nextLine();
116                 rs_3025.cariPasien_3025(cari_3025);
117                 break;
118             case 5:
119                 rs_3025.cekStatus_3025();
120                 break;
121             case 6:
122                 System.out.println("Terima kasih.");
123                 break;
124             default:
125                 System.out.println("Pilihan tidak valid!");
126         }
127     } while (pilih_3025 != 6);
128 }
129 }
130 }
131 }
132 }
133 }
134 }
135 }
136 }
137 }
138 }
139 }
140 }
141 }
142 }
143 }

```

3. Gambar output code program

Problems Javadoc Declaration Console X Progress Git Staging
<terminated> DriverRumahSakit_2511533025 [Java Application] /Applications/Eclipse.app/Contents/Resources/Java/bin/java -Xmx1024m -Xms128m -Djava.library.path=...
=== Antrian Rumah Sakit NIM: 2511533025 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Fajar
Masukkan Keluhan : Diare
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533025 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Arkan
Masukkan Keluhan : Pusing
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511533025 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Andre
Masukkan Keluhan : Darah Rendah
Pasien berhasil didaftarkan! Nomor Antrian: 3

```
==== Antrian Rumah Sakit NIM: 2511533025 ===
```

```
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
No: 1 | Nama: Fajar | Keluhan: Diare
No: 2 | Nama: Arkan | Keluhan: Pusing
No: 3 | Nama: Andre | Keluhan: Darah Rendah
```

```
==== Antrian Rumah Sakit NIM: 2511533025 ===
```

```
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama : Fajar
Pasien ditemukan!
No Antrian: 1
```

```
==== Antrian Rumah Sakit NIM: 2511533025 ===
```

```
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien: 3
Pasien terdepan: Fajar
```

```

=== Antrian Rumah Sakit NIM: 2511533025 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Pasien dipanggil:
Nama : Fajar
Keluhan : Diare
Nomor Antrian : 1

=== Antrian Rumah Sakit NIM: 2511533025 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Pasien dipanggil:
Nama : Arkan
Keluhan : Pusing
Nomor Antrian : 2

=== Antrian Rumah Sakit NIM: 2511533025 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Pasien dipanggil:
Nama : Andre
Keluhan : Darah Rendah
Nomor Antrian : 3

=== Antrian Rumah Sakit NIM: 2511533025 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Terima kasih.

```

5. Penjelasan sederhana tentang kode program

Program tersebut merupakan simulasi antrian pasien rumah sakit yang dibuat menggunakan konsep Single Linked List. Terdapat dua kelas utama, yaitu kelas Pasien sebagai ADT (Abstract Data Type) dan kelas Rumah Sakit sebagai pengelola antrian. Pada kelas Pasien, setiap objek merepresentasikan satu pasien yang memiliki data berupa nama, keluhan penyakit, nomor antrian, serta pointer (next) yang menunjuk ke

pasien berikutnya. Pointer ini digunakan untuk menghubungkan setiap node sehingga membentuk sebuah linked list.

Pada kelas Rumah Sakit, program mengelola alur antrian menggunakan prinsip FIFO (First In First Out), yaitu pasien yang datang lebih dulu akan dilayani lebih dulu. Saat pasien didaftarkan, data pasien akan ditambahkan di bagian akhir (tail) linked list dan nomor antrian akan bertambah otomatis. Ketika pasien dipanggil, data pasien terdepan (head) akan dihapus dan head akan berpindah ke node berikutnya. Program juga menyediakan fitur untuk menampilkan seluruh antrian dengan menelusuri linked list dari awal hingga akhir, mencari pasien berdasarkan nama tanpa membedakan huruf besar dan kecil, serta mengecek status antrian seperti jumlah pasien dan siapa yang berada di posisi terdepan.

Secara keseluruhan, program ini menunjukkan bagaimana struktur data linked list bekerja secara dinamis menggunakan referensi antar node, sehingga data dapat ditambah dan dihapus tanpa batas ukuran seperti pada array.